

Lab 5 – Đồ Thị

1. Mục tiêu

Kết thúc bài thực hành, sinh viên có khả năng:

- Biểu diễn đồ thị bằng dictionary và list trong Python
- Cài đặt thuật toán BFS và DFS
- Phát hiện chu trình trong đồ thị vô hướng và có hướng
- Tìm connected components sử dụng BFS/DFS
- Cài đặt Topological Sort bằng DFS và Kahn's Algorithm
- Áp dụng giải quyết bài toán thực tế (Course Schedule)

2. Nội dung

Bài 1 – Graph Traversal cơ bản

Yêu cầu chung

Tạo file lab5_bai1.py, viết 4 hàm sau đây. Mỗi hàm phải có:

- Comment ghi rõ mục đích và cách hoạt động
- Test với ít nhất 2 trường hợp
- In kết quả rõ ràng

Hàm 1 – Xây dựng đồ thị từ danh sách cạnh

Yêu cầu: Viết hàm `build_graph(edges, directed=False)` xây dựng đồ thị từ danh sách cạnh

Hướng dẫn chi tiết:

Bước 1: Phân tích bài toán

- Input:
 - edges: List các cạnh [(u, v), ...]
 - directed: Boolean (True = có hướng, False = vô hướng)
- Output: Dictionary biểu diễn đồ thị

Bước 2: Chiến lược

- Dùng dictionary với key là đỉnh, value là list các đỉnh kề
- Với đồ thị vô hướng: thêm cả $u \rightarrow v$ và $v \rightarrow u$
- Với đồ thị có hướng: chỉ thêm $u \rightarrow v$

Bước 3: Code hoàn chỉnh

```
def build_graph(edges, directed=False):
    """
    Xây dựng đồ thị từ danh sách cạnh

    Input:
        edges: list của các tuple (u, v) đại diện cho cạnh
        directed: True nếu đồ thị có hướng, False nếu vô hướng

    Output:
        graph: dictionary với key là đỉnh, value là list các đỉnh kề
    """
    graph = {}

    # Duyệt qua từng cạnh
    for u, v in edges:
        # Khởi tạo đỉnh u nếu chưa có
        if u not in graph:
```

```

        graph[u] = []
        # Khởi tạo đỉnh v nếu chưa có
        if v not in graph:
            graph[v] = []

        # Thêm cạnh  $u \rightarrow v$ 
        graph[u].append(v)

        # Nếu vô hướng, thêm cạnh  $v \rightarrow u$ 
        if not directed:
            graph[v].append(u)

    return graph

# Test case 1: Đồ thị vô hướng
print("=== Test 1: Đồ thị vô hướng ===")
edges1 = [
    ('A', 'B'),
    ('A', 'C'),
    ('B', 'D'),
    ('C', 'D'),
    ('D', 'E')
]

graph1 = build_graph(edges1, directed=False)
print("Đồ thị vô hướng:")
for vertex, neighbors in graph1.items():
    print(f" {vertex}: {neighbors}")

```

```
# Output mong đợi:
# A: ['B', 'C']
# B: ['A', 'D']
# C: ['A', 'D']
# D: ['B', 'C', 'E']
# E: ['D']

# Test case 2: Đồ thị có hướng
print("\n=== Test 2: Đồ thị có hướng ===")
edges2 = [
    ('A', 'B'),
    ('A', 'C'),
    ('B', 'D'),
    ('C', 'D'),
    ('D', 'E')
]

graph2 = build_graph(edges2, directed=True)
print("Đồ thị có hướng:")
for vertex, neighbors in graph2.items():
    print(f" {vertex}: {neighbors}")

# Output mong đợi:
# A: ['B', 'C']
# B: ['D']
# C: ['D']
# D: ['E']
# E: []
```

Bước 4: Giải thích

- **Khởi tạo đỉnh:** Đảm bảo mọi đỉnh đều có entry trong graph (kể cả đỉnh không có cạnh ra)
- **Vô hướng vs Có hướng:**
 - Vô hướng: mỗi cạnh tạo 2 kết nối
 - Có hướng: mỗi cạnh tạo 1 kết nối
- **Độ phức tạp:** $O(E)$ với E là số cạnh

Hàm 2 – BFS (Breadth-First Search)

Yêu cầu: Viết hàm `bfs(graph, start)` duyệt đồ thị theo chiều rộng

Hướng dẫn chi tiết:

Bước 1: Phân tích bài toán

- Input:
 - `graph`: Dictionary biểu diễn đồ thị
 - `start`: Đỉnh bắt đầu
- Output: List các đỉnh theo thứ tự duyệt BFS

Bước 2: Chiến lược BFS

- Dùng **Queue** (FIFO - First In First Out)
- Dùng **Set** để đánh dấu visited (tránh thăm lại)
- Duyệt theo từng lớp (level by level)

Bước 3: Code hoàn chỉnh

```
from collections import deque
```

```
def bfs(graph, start):
```

```
    """
```

BFS - Duyệt đồ thị theo chiều rộng

Chiến lược:

1. Dùng Queue để lưu các đỉnh cần thăm
2. Dùng Set để đánh dấu đã thăm
3. Lặp: Lấy đỉnh từ queue, thăm, thêm neighbors chưa thăm vào queue

Input:

graph: dictionary biểu diễn đồ thị

start: đỉnh bắt đầu

Output:

result: list các đỉnh theo thứ tự duyệt

```
    """
```

```
    # Khởi tạo
```

```
    visited = set()
```

```
    queue = deque([start])
```

```
    visited.add(start)
```

```
    result = []
```

```
    # BFS loop
```

```
    while queue:
```

```
        # Lấy đỉnh từ đầu queue (FIFO)
```

```
        vertex = queue.popleft()
```

```
        result.append(vertex)
```

```
        # Duyệt các đỉnh kề
```

```

    for neighbor in graph[vertex]:
        # Nếu chưa thăm, đánh dấu và thêm vào queue
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)

    return result

# Test case 1
print("\n=== Test BFS ===")
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

result = bfs(graph, 'A')
print(f"BFS từ A: {result}")
# Output: ['A', 'B', 'C', 'D', 'E', 'F']
# Giải thích: Thăm A trước, sau đó thăm lớp 1 (B, C), rồi lớp 2 (D, E, F)

# Test case 2: Bắt đầu từ đỉnh khác
result2 = bfs(graph, 'D')
print(f"BFS từ D: {result2}")
# Output: ['D', 'B', 'A', 'E', 'C', 'F']

```

Bước 4: Giải thích

- **Queue:** Đảm bảo duyệt theo từng lớp (đỉnh thêm trước được xử lý trước)
- **Visited:** Tránh thăm lại đỉnh (tránh vòng lặp vô hạn)
- **Thời điểm đánh dấu:** Đánh dấu visited khi **thêm vào queue**, không phải khi lấy ra
- **Độ phức tạp:** $O(V + E)$ với V đỉnh, E cạnh

Hàm 3 – DFS đệ quy (Depth-First Search)

Yêu cầu: Viết hàm `dfs_recursive(graph, start)` duyệt đồ thị theo chiều sâu bằng đệ quy

Hướng dẫn chi tiết:

Bước 1: Phân tích bài toán

- Input:
 - `graph`: Dictionary biểu diễn đồ thị
 - `start`: Đỉnh bắt đầu
- Output: List các đỉnh theo thứ tự duyệt DFS

Bước 2: Chiến lược DFS

- Dùng **Đệ quy** (hoặc Stack)
- Đi sâu vào một nhánh trước khi quay lui (backtrack)
- Dùng Set để đánh dấu visited

Bước 3: Code hoàn chỉnh

```
def dfs_recursive(graph, start, visited=None, result=None):
```

```
    """
```

```
    DFS - Duyệt đồ thị theo chiều sâu (đệ quy)
```

```
    Chiến lược:
```


1. Đánh dấu đỉnh hiện tại là visited
2. Thêm vào kết quả
3. Với mỗi neighbor chưa visited, đệ quy DFS

Input:

graph: dictionary biểu diễn đồ thị

start: đỉnh bắt đầu

visited: set các đỉnh đã thăm (dùng nội bộ)

result: list kết quả (dùng nội bộ)

Output:

result: list các đỉnh theo thứ tự duyệt

"""

Khởi tạo lần đầu

if visited is None:

visited = set()

if result is None:

result = []

Đánh dấu đỉnh hiện tại

visited.add(start)

result.append(start)

Đệ quy với các đỉnh kề chưa thăm

for neighbor in graph[start]:

if neighbor not in visited:

dfs_recursive(graph, neighbor, visited, result)

return result

```

# Test case 1
print("\n=== Test DFS Recursive ===")
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

result = dfs_recursive(graph, 'A')
print(f"DFS từ A: {result}")
# Output: ['A', 'B', 'D', 'E', 'F', 'C']
# Giải thích: Đi sâu A→B→D (không còn đường), quay về B→E→F, cuối cùng C

# Test case 2
result2 = dfs_recursive(graph, 'C')
print(f"DFS từ C: {result2}")

```

Bước 4: Giải thích

- **Đệ quy:** Tự động sử dụng call stack thay vì stack tường minh
- **Thứ tự:** Đi sâu vào neighbor đầu tiên trước khi xét neighbor tiếp theo
- **Visited:** Cần truyền qua các lần gọi đệ quy để tránh thăm lại
- **Độ phức tạp:** $O(V + E)$, không gian $O(V)$ cho call stack

Hàm 4 – Connected Components

Yêu cầu: Viết hàm `count_connected_components(graph)` đếm số thành phần liên thông

Hướng dẫn chi tiết:

Bước 1: Phân tích bài toán

- Input: Dictionary biểu diễn đồ thị
- Output:
 - Số lượng connected components
 - List các components (mỗi component là một list đỉnh)

Bước 2: Chiến lược

- Duyệt tất cả các đỉnh
- Với mỗi đỉnh chưa visited: chạy BFS/DFS từ đỉnh đó
- Mỗi lần chạy BFS/DFS = 1 component mới

Bước 3: Code hoàn chỉnh

```
def count_connected_components(graph):  
    """  
    Đếm số connected components trong đồ thị  
  
    Chiến lược:  
    1. Duyệt tất cả các đỉnh  
    2. Với mỗi đỉnh chưa visited:  
        - Chạy BFS từ đỉnh đó  
        - Tất cả đỉnh được thăm tạo thành 1 component  
    3. Số lần chạy BFS = số components  
  
    Input:  
    graph: dictionary biểu diễn đồ thị
```

Output:

count: số lượng components

components: list các components

"""

visited = set()

components = []

def bfs_component(start):

"""BFS helper để tìm một component"""

queue = deque([start])

visited.add(start)

component = []

while queue:

vertex = queue.popleft()

component.append(vertex)

for neighbor in graph[vertex]:

if neighbor not in visited:

visited.add(neighbor)

queue.append(neighbor)

return component

Duyệt tất cả các đỉnh

for vertex in graph:

if vertex not in visited:

Tìm component chứa vertex

```
        component = bfs_component(vertex)
        components.append(component)

    return len(components), components

# Test case 1: Đồ thị có 1 component
print("\n=== Test Connected Components ===")
print("Test 1: 1 component")
graph1 = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A'],
    'D': ['B']
}

count1, comps1 = count_connected_components(graph1)
print(f"Số components: {count1}")
for i, comp in enumerate(comps1, 1):
    print(f"Component {i}: {comp}")
# Output: 1 component chứa tất cả đỉnh

# Test case 2: Đồ thị có nhiều components
print("\nTest 2: 3 components")
graph2 = {
    'A': ['B'],
    'B': ['A'],
    'C': ['D', 'E'],
    'D': ['C'],
```

```

'E': ['C'],
'F': [],
}

count2, comps2 = count_connected_components(graph2)
print(f"Số components: {count2}")
for i, comp in enumerate(comps2, 1):
    print(f"Component {i}: {comp}")
# Output:
# Component 1: ['A', 'B']
# Component 2: ['C', 'D', 'E']
# Component 3: ['F']

```

Bước 4: Giải thích

- **Tại sao đúng:** Mỗi lần BFS thăm hết một component liên thông
- **Đỉnh cô lập:** Đỉnh không có cạnh nào cũng là 1 component
- **Độ phức tạp:** $O(V + E)$ - mỗi đỉnh và cạnh được xét đúng 1 lần
- **Ứng dụng:** Phát hiện mạng lưới con, nhóm bạn bè, phân vùng dữ liệu

Bài 2 – Cycle Detection

Trong bài này, sinh viên cài đặt thuật toán phát hiện chu trình cho cả đồ thị vô hướng và có hướng.

File: lab5_bai2.py

Phần A – Phát hiện chu trình trong đồ thị vô hướng (có code khung)

Yêu cầu: Viết hàm `has_cycle_undirected(graph)` phát hiện chu trình trong đồ thị vô hướng

Code khung:

```
def has_cycle_undirected(graph):
    """
    Phát hiện chu trình trong đồ thị vô hướng

    Chiến lược:
    - Dùng DFS với tham số parent
    - Nếu gặp đỉnh đã visited VÀ không phải parent → có chu trình

    Tại sao cần kiểm tra parent?
    - Đồ thị vô hướng: A-B nghĩa là có  $A \rightarrow B$  và  $B \rightarrow A$ 
    - Khi đang ở A, đi tới B, sau đó B có cạnh về A
    - Đây không phải chu trình, chỉ là cạnh vừa đi!
    """
    visited = set()

    def dfs(vertex, parent):
        """
        DFS helper
        vertex: đỉnh hiện tại
        parent: đỉnh cha (đỉnh đã đi qua để tới vertex)
        """
        # TODO: Đánh dấu vertex là visited
        visited.add(vertex)

        # TODO: Duyệt các neighbors
        for neighbor in graph[vertex]:
            if neighbor not in visited:
                # TODO: Đệ quy với neighbor
                if dfs(neighbor, vertex):
                    return True
            # Nếu đã visited nhưng không phải parent thì có chu trình
            elif neighbor == parent:
                return True
        return False

    return dfs(None, None)
```

```

        return True

    elif neighbor != parent:
        # TODO: Gặp đỉnh đã visited và không phải cha
        # → Có chu trình!
        return True

    return False

# TODO: Duyệt tất cả các đỉnh (đồ thị có thể không liên thông)
for vertex in graph:
    if vertex not in visited:
        if dfs(vertex, None):
            return True

return False

# Test cases
print("=== Test Cycle Detection - Undirected ===")

# Test 1: Có chu trình
graph1 = {
    0: [1, 2],
    1: [0, 3],
    2: [0, 3],
    3: [1, 2]
}

print(f"Test 1 (có chu trình): {has_cycle_undirected(graph1)}") # True

```



```

# Test 2: Không có chu trình (cây)
graph2 = {
    0: [1, 2],
    1: [0, 3],
    2: [0],
    3: [1]
}

print(f"Test 2 (không có chu trình): {has_cycle_undirected(graph2)}") # False

# Test 3: Đồ thị không liên thông với chu trình
graph3 = {
    0: [1],
    1: [0],
    2: [3, 4],
    3: [2, 4],
    4: [2, 3]
}

print(f"Test 3 (nhiều components, có chu trình): {has_cycle_undirected(graph3)}") # True

```

Yêu cầu sinh viên:

1. Hoàn thành các TODO trong code
2. Giải thích tại sao cần kiểm tra `neighbor != parent`
3. Test với ít nhất 3 test case khác nhau
4. Vẽ đồ thị ra giấy và trace thuật toán

Phần B – Phát hiện chu trình trong đồ thị có hướng

Yêu cầu: Viết hàm `has_cycle_directed(graph)` phát hiện chu trình trong đồ thị có hướng

Gợi ý chiến lược:

Three-Color Approach:

- **WHITE (0):** Chưa thăm
- **GRAY (1):** Đang xử lý (trong recursion stack)
- **BLACK (2):** Đã xử lý xong

Quy tắc: Nếu gặp đỉnh GRAY → có chu trình!

Code khung:

```
def has_cycle_directed(graph):  
    """  
    Phát hiện chu trình trong đồ thị có hướng  
    Sử dụng three-color approach  
  
    WHITE: Chưa thăm  
    GRAY: Đang xử lý (trong recursion stack)  
    BLACK: Đã xử lý xong  
  
    Nếu gặp đỉnh GRAY → có back edge → có chu trình!  
    """  
    WHITE, GRAY, BLACK = 0, 1, 2  
  
    # TODO: Khởi tạo color cho tất cả đỉnh là WHITE  
    color = {vertex: WHITE for vertex in graph}  
  
    def dfs(vertex):  
        """DFS helper với three-color"""  
        # TODO: Đánh dấu đỉnh đang xử lý (GRAY)  
        color[vertex] = GRAY
```

```
# TODO: Duyệt các neighbors
for neighbor in graph[vertex]:
    # Nếu neighbor đang GRAY → trong recursion stack → có chu trình
    if color[neighbor] == GRAY:
        return True

    # Nếu neighbor là WHITE → chưa thăm
    if color[neighbor] == WHITE:
        if dfs(neighbor):
            return True

# TODO: Xử lý xong đỉnh này (BLACK)
color[vertex] = BLACK
return False

# TODO: Duyệt tất cả các đỉnh
for vertex in graph:
    if color[vertex] == WHITE:
        if dfs(vertex):
            return True

return False

# Test cases
print("\n=== Test Cycle Detection - Directed ===")

# Test 1: Có chu trình
graph1 = {
```

```

'A': ['B'],
'B': ['C'],
'C': ['A']
}
print(f"Test 1 (có chu trình  $A \rightarrow B \rightarrow C \rightarrow A$ ): {has_cycle_directed(graph1)})" # True

# Test 2: DAG (không có chu trình)
graph2 = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['D'],
    'D': []
}
print(f"Test 2 (DAG): {has_cycle_directed(graph2)})" # False

# Test 3: Cross edge (không phải chu trình)
graph3 = {
    'A': ['B'],
    'B': ['C'],
    'C': [],
    'D': ['C']
}
print(f"Test 3 (cross edge, không có chu trình): {has_cycle_directed(graph3)})" # False

```

Yêu cầu sinh viên:

1. Hoàn thành các TODO
2. Giải thích tại sao cần 3 trạng thái (WHITE/GRAY/BLACK)
3. Giải thích sự khác biệt với đồ thị vô hướng
4. Test với các trường hợp: có chu trình, DAG, cross edge

Phần C – So sánh và phân tích

Yêu cầu: Viết hàm `compare_cycle_detection()` so sánh hai phương pháp

Gợi ý:

```
def compare_cycle_detection():
    """
    So sánh cycle detection giữa đồ thị vô hướng và có hướng
    """
    print("\n" + "="*60)
    print("SO SÁNH CYCLE DETECTION")
    print("="*60)

    # TODO: Test case đồ thị vô hướng
    print("\n[1] ĐỒ THỊ VÔ HƯỚNG:")
    undirected = {
        # Tạo đồ thị test
    }
    # Test và in kết quả

    # TODO: Test case đồ thị có hướng
    print("\n[2] ĐỒ THỊ CÓ HƯỚNG:")
    directed = {
        # Tạo đồ thị test
    }
    # Test và in kết quả

    # TODO: So sánh
    print("\n[3] SO SÁNH:")
    print("Vô hướng: Kiểm tra parent")
```

```
print("Có hướng: Three-color (WHITE/GRAY/BLACK)")
print("Cả hai: Độ phức tạp  $O(V + E)$ ")
```

Sinh viên tự hoàn thành.

Bài 3 – Topological Sort

Trong bài này, sinh viên cài đặt Topological Sort và áp dụng vào bài toán Course Schedule.

File: lab5_bai3.py

Phần A – Topological Sort với DFS

Yêu cầu: Viết hàm `topological_sort_dfs(graph)` sắp xếp topo bằng DFS

Gợi ý chiến lược:

Post-Order DFS:

- Thêm đỉnh vào kết quả SAU KHI xử lý hết các neighbors
- Đảm bảo con cháu xuất hiện trước cha

Code khung:

```
def topological_sort_dfs(graph):
```

```
    """
```

```
    Topological Sort sử dụng DFS (post-order)
```

```
    Chiến lược:
```

1. Kiểm tra có chu trình không (chỉ DAG mới có topo sort)
2. DFS từ mọi đỉnh chưa visited
3. Thêm đỉnh vào stack SAU KHI xử lý hết neighbors (post-order)
4. Return stack đảo ngược

Return: List theo thứ tự topo, hoặc None nếu có chu trình

```
"""
```

```
# TODO: Kiểm tra chu trình trước
```

```
# if has_cycle_directed(graph):
```

```
#     return None
```

```
visited = set()
```

```
stack = []
```

```
def dfs(vertex):
```

```
    """DFS helper với post-order"""
```

```
    # TODO: Đánh dấu visited
```

```
    # TODO: Duyệt neighbors
```

```
    # TODO: Thêm vào stack (post-order)
```

```
    pass
```

```
# TODO: DFS từ mọi đỉnh chưa visited
```

```
# TODO: Return stack đảo ngược
```

```
pass
```

Sinh viên tự hoàn thành.

Phần B – Topological Sort với Kahn's Algorithm

Yêu cầu: Viết hàm `topological_sort_kahn(graph)` sử dụng BFS + in-degree

Gợi ý chiến lược:

Kahn's Algorithm:

1. Tính in-degree (bậc vào) của mỗi đỉnh
2. Thêm các đỉnh có in-degree = 0 vào queue
3. Lặp:
 - Lấy đỉnh từ queue, thêm vào kết quả
 - Giảm in-degree của neighbors
 - Nếu in-degree = 0 $\rightarrow$ thêm vào queue
4. Nếu len(result) < V $\rightarrow$ có chu trình

Code khung:

```
from collections import deque

def topological_sort_kahn(graph):
    """
    Topological Sort sử dụng Kahn's Algorithm (BFS + in-degree)

    Return: List theo thứ tự topo, hoặc None nếu có chu trình
    """
    # TODO: Tính in-degree cho mỗi đỉnh
    in_degree = {}

    # TODO: Thêm đỉnh có in-degree = 0 vào queue
    queue = deque()

    # TODO: BFS loop
    result = []
```



```
# TODO: Kiểm tra chu trình
if len(result) != len(graph):
    return None # Có chu trình

return result
```

Sinh viên tự nghiên cứu và cài đặt.

Phần C – Course Schedule

Yêu cầu: Áp dụng cycle detection và topological sort vào bài toán thực tế

Bài toán 1: Course Schedule

```
def can_finish(num_courses, prerequisites):
```

```
    """
```

Kiểm tra có thể hoàn thành tất cả môn học không?

Input:

num_courses: số môn học (0 đến n-1)

prerequisites: list [[a, b]] nghĩa là phải học b trước a

Output:

True nếu có thể học hết, False nếu có chu trình

Gợi ý:

1. Xây dựng đồ thị từ prerequisites
2. Kiểm tra chu trình
3. Có chu trình → False, không có → True

```
    """
```

```
# TODO: Xây dựng đồ thị
```

```
# TODO: Kiểm tra chu trình
```

```
pass
```

Bài toán 2: Course Schedule II

```
def find_order(num_courses, prerequisites):
```

```
    """
```

Tìm thứ tự học hợp lệ

Input: giống Course Schedule

Output: List thứ tự học, hoặc [] nếu không thể

Gợi ý:

1. Xây dựng đồ thị
2. Topological sort
3. Có chu trình $\rightarrow$ [], không có $\rightarrow$ return order

```
    """
```

```
# TODO: Xây dựng đồ thị
```

```
# TODO: Topological sort
```

```
pass
```

Test cases:

```
print("\n=== Test Course Schedule ===")
```

```
# Test 1: Có thể hoàn thành
```

```
n1 = 4
```

```
prereqs1 = [[1, 0], [2, 0], [3, 1], [3, 2]]
print(f"Test 1: {can_finish(n1, prereqs1)}") # True
print(f"Order: {find_order(n1, prereqs1)}") # [0, 1, 2, 3] hoặc [0, 2, 1, 3]

# Test 2: Có chu trình
n2 = 2
prereqs2 = [[1, 0], [0, 1]]
print(f"Test 2: {can_finish(n2, prereqs2)}") # False
print(f"Order: {find_order(n2, prereqs2)}") # []
```

Sinh viên tự hoàn thành và test.